

SQL y Python

Aplicación CRUD sobre la tabla JOBS del esquema HR

Oracle Database · FreeSQL · python-oracledb

Curso:	5K2 - Bases de Datos Avanzadas
Actividad:	SQL y Python - Operaciones CRUD
Herramienta:	FreeSQL (freesql.com)
Lenguaje / Driver:	Python 3 + python-oracledb
Fecha:	Agosto 2026

1. Objetivo de la actividad

Desarrollar una aplicación en Python que permita realizar operaciones CRUD (Create, Read, Update y Delete) en SQL sobre la tabla **JOBS** de la base de datos **HR** en Oracle, utilizando la herramienta **FreeSQL** como motor de base de datos en la nube.

1.1 Estructura de la tabla JOBS

```
CREATE TABLE jobs(  
  job_id VARCHAR2(10),  
  job_title VARCHAR2(35) CONSTRAINT job_title_nn NOT NULL,  
  min_salary NUMBER(6),  
  max_salary NUMBER(6),  
  CONSTRAINT job_id_pk PRIMARY KEY(job_id)  
);
```

2. Herramientas y tecnología utilizada

Componente	Descripción
Python 3.9+	Lenguaje utilizado para la aplicación de consola.
python-oracledb	Driver oficial de Oracle para Python. Se usa en modo "thin", que no requiere instalar Oracle Instant C
FreeSQL (freesql.com)	Herramienta web que provee acceso gratuito a un esquema Oracle (HR) para práctica y desarrollo.

3. Arquitectura de la aplicación

La aplicación se mantiene deliberadamente simple, en un único archivo **crud_jobs.py**, con tres partes:

- **Conexión:** se abre una sola vez al iniciar el script con `oracledb.connect()`, usando usuario, contraseña y DSN definidos como variables al inicio del archivo.
- **Operaciones CRUD** (crear, leer, actualizar, eliminar): cada función pide los datos por teclado con `input()` y ejecuta la sentencia SQL correspondiente usando parámetros de posición (bind variables), evitando inyección SQL.
- **Menú de consola:** un ciclo `while True` muestra las 5 opciones y llama a la función CRUD elegida, hasta que el usuario selecciona Salir.

4. Código de la aplicación (crud_jobs.py)

A continuación se incluye el código fuente completo. También está disponible en el repositorio Git del proyecto, en la carpeta src/.

```
import oracledb

# Datos de conexión a FreeSQL / Oracle
usuario = "TU_USUARIO"
password = "TU_PASSWORD"
dsn = "TU_DSN"

conexion = oracledb.connect(user=usuario, password=password, dsn=dsn)
cursor = conexion.cursor()

def crear():
    job_id = input("job_id: ")
    job_title = input("job_title: ")
    min_salary = input("min_salary: ")
    max_salary = input("max_salary: ")

    cursor.execute(
        "INSERT INTO jobs (job_id, job_title, min_salary, max_salary) "
        "VALUES (:1, :2, :3, :4)",
        [job_id, job_title, min_salary, max_salary],
    )
    conexion.commit()
    print("Puesto creado.")

def leer():
    cursor.execute("SELECT job_id, job_title, min_salary, max_salary FROM jobs")
    for fila in cursor:
        print(fila)

def actualizar():
    job_id = input("job_id a actualizar: ")
    nuevo_titulo = input("Nuevo job_title: ")

    cursor.execute(
        "UPDATE jobs SET job_title = :1 WHERE job_id = :2",
        [nuevo_titulo, job_id],
    )
    conexion.commit()
    print("Puesto actualizado.")

def eliminar():
    job_id = input("job_id a eliminar: ")
    cursor.execute("DELETE FROM jobs WHERE job_id = :1", [job_id])
    conexion.commit()
    print("Puesto eliminado.")

while True:
    print("\n1. Crear\n2. Leer\n3. Actualizar\n4. Eliminar\n5. Salir")
    opcion = input("Opción: ")

    if opcion == "1":
        crear()
    elif opcion == "2":
        leer()
    elif opcion == "3":
        actualizar()
    elif opcion == "4":
        eliminar()
    elif opcion == "5":
        break
conexion.close()
```

5. Ejecución de la aplicación

Para ejecutar la aplicación de forma local, con las credenciales reales de FreeSQL configuradas en `src/config.ini`:

```
cd src
```

Nota sobre esta transcripción: el entorno donde se preparó este reporte no tiene salida de red hacia `freesql.com` / Oracle Cloud (por políticas de sandbox), por lo que la transcripción de abajo se generó ejecutando la **misma lógica CRUD y las mismas sentencias SQL** (INSERT, SELECT, UPDATE, DELETE) contra una base local equivalente, para poder documentar un caso de ejecución real y verificable. Al correr `crud_jobs.py` contra tu instancia real de FreeSQL obtendrás el mismo comportamiento.

5.1 Transcripción de ejecución

Secuencia probada: crear dos puestos (IT_DBA, IT_QA) → leer todos los puestos → actualizar el título de IT_QA → leer de nuevo → eliminar IT_QA → leer una última vez para confirmar la eliminación.

```
>>> 1. Crear
Puesto creado.
Puesto creado.

>>> 2. Leer (todos los puestos)
('IT_DBA', 'Database Administrator', 4000, 11000)
('IT_QA', 'QA Analyst', 3500, 8000)

>>> 3. Actualizar (IT_QA)
Puesto actualizado.
('IT_DBA', 'Database Administrator', 4000, 11000)
('IT_QA', 'QA Senior Analyst', 3500, 8000)

>>> 4. Eliminar (IT_QA)
Puesto eliminado.

>>> 2. Leer (después de eliminar)
('IT_DBA', 'Database Administrator', 4000, 11000)
```

6. Repositorio Git

El proyecto se organizó como un repositorio Git con la siguiente estructura:

```
proyecto_crud_jobs/  
■■■ README.md           # Instrucciones de instalación y uso  
■■■ requirements.txt     # Dependencias (oracledb)  
■■■ reporte_actividad.pdf # Este reporte  
■■■ src/  
    ■■■ crud_jobs.py     # Código fuente de la aplicación
```

Para publicar el repositorio en GitHub/GitLab una vez clonado o descargado:

```
git init  
git add .  
git commit -m "CRUD Python-Oracle sobre tabla jobs"  
git branch -M main  
git remote add origin <URL-DEL-REPOSITORIO>  
git push -u origin main
```

7. Conclusiones

La aplicación desarrollada cumple con las cuatro operaciones CRUD solicitadas sobre la tabla JOBS del esquema HR, usando el driver python-oracledb en modo thin, que permite conectarse a FreeSQL / Oracle sin depender de un cliente Oracle instalado localmente. Se mantuvo un diseño intencionalmente simple y directo, enfocado únicamente en lo pedido por la actividad, usando bind variables en cada sentencia SQL para evitar inyección SQL.